

# Fundamentos de POO: De la Lógica de Scripts a la Arquitectura de Objetos

## 1. Introducción al Paradigma: El Cambio de Mentalidad

---

Hasta ahora, han trabajado con **Programación Estructurada**. Han creado funciones que toman datos, hacen algo con ellos y retornan un resultado. Este enfoque es excelente para scripts pequeños, validaciones o manipulación directa del DOM. Sin embargo, cuando el proyecto crece a cientos o miles de líneas, mantener estas funciones sueltas se vuelve caótico.

Imaginen construir una casa: la programación estructurada es como tener un taladro, martillos y clavos sueltos en una caja. Funcionan, pero cada vez que quieres construir una pared, tienes que buscar las herramientas y recordar cómo encajan.

La **Programación Orientada a Objetos (POO)** cambia las reglas. En lugar de pensar en "funciones que actúan sobre datos", pensamos en **entidades con vida propia**. Dejamos de tener datos sueltos y funciones sueltas, para agruparlos en "objetos" que contienen tanto sus datos (atributos) como su comportamiento (métodos).

| Cómo se hacía antes (Funciones sueltas)   | Cómo se hace ahora (POO)                                        |
|-------------------------------------------|-----------------------------------------------------------------|
| Datos y funciones separados.              | Datos y funciones unidos en un objeto.                          |
| <code>procesarPago(usuario, monto)</code> | <code>usuario.realizarPago(monto)</code>                        |
| Se pasa el estado como parámetro.         | El objeto <i>conoce</i> su propio estado ( <code>this</code> ). |
| Dificultad para mantener el estado.       | El estado está encapsulado dentro del objeto.                   |

**Nota del Instructor:**

*Dejen de pensar en "funciones que hacen cosas" y empiecen a pensar en "objetos que son cosas". El código no es una lista de instrucciones; es una simulación de entidades que interactúan.*

## 2. El Plano y la Obra (Clases e Instancias)

Si la POO trata sobre objetos, ¿cómo creamos muchos objetos que sean iguales en estructura pero diferentes en valores?

Aquí es donde aparece la **Clase**. Una clase es el *plano arquitectónico*. Define qué propiedades (datos) y qué métodos (comportamientos) tendrá el objeto, pero no consume memoria hasta que lo usamos.

La **Instancia** (u Objeto) es la *obra construida*. Es el resultado tangible creado a partir de ese plano.

```
// 1. EL PLANO (Class)
class NaveEspacial {
    // Esto es una propiedad del plano
    tipo = "Exploración";

    // Esto es un método
    despegar() {
        console.log("Iniciando secuencia de despegue...");
    }
}

// 2. LA OBRA (Instancia)
// Usamos 'new' para crear una copia física del plano en memoria
const apolo = new NaveEspacial();
const artemis = new NaveEspacial();

apolo.despegar(); // "Iniciando secuencia de despegue..."
console.log(apolo.tipo); // "Exploración"

// Aunque vienen del mismo plano, son objetos independientes
console.log(apolo === artemis); // false
```

### Nota del Instructor:

*class define la estructura; new reserva el espacio en memoria y ejecuta el constructor. Si no usas new, la máquina no sabe qué "fábrica" debe activar para crear el objeto.*

## 3. El Método Constructor: Inicialización y

**this**

---

El **constructor** es un método especial que se ejecuta automáticamente cuando usamos **new**. Su trabajo es *inicializar* al objeto. Sin él, todos los objetos tendrían los mismos valores por defecto.

La palabra clave **this** es una de las más confusas pero cruciales. **this** es una **referencia al objeto que está siendo creado (o que está ejecutando el método en**

**ese momento).** Es como decir "yo mismo" o "mi propio".

```
class Usuario {  
    // Constructor: Recibe parámetros para personalizar la instancia  
    constructor(nombre, edad) {  
        // 'this.nombre' es la propiedad DEL OBJETO  
        // 'nombre' es el parámetro que llegó  
        this.nombre = nombre;  
        this.edad = edad;  
        this.fechaRegistro = new Date(); // Valor por defecto  
    }  
  
    mostrarPerfil() {  
        // Aquí 'this' se refiere al objeto específico que llama al método  
        return `Usuario: ${this.nombre}, Edad: ${this.edad}`;  
    }  
}  
  
const ana = new Usuario("Ana López", 28);  
const luis = new Usuario("Luis Pérez", 32);  
  
console.log(ana.mostrarPerfil()); // Usuario: Ana López, Edad: 28  
console.log(luis.mostrarPerfil()); // Usuario: Luis Pérez, Edad: 32
```

### Nota del Instructor:

*Sin `this`, no podríamos diferenciar los datos de Ana de los de Luis. `this` actúa como la cédula de identidad del objeto. El constructor es tu oportunidad de configurar el objeto correctamente desde su nacimiento.*

## 4. Los 4 Pilares de la POO

---

### 4.1 Abstracción

**Simplificar la realidad.** Cuando modelamos un objeto, no necesitamos todos los detalles del mundo real, solo los relevantes para nuestro sistema.

*Ejemplo:* Para un sistema de gestión de una biblioteca, de un "Libro" solo nos importa `titulo` , `autor` , `ISBN` , `prestado` . No nos importa el número de páginas de la tapa ni el peso en gramos.

## 4.2 Encapsulamiento

**Proteger el estado interno.** En JavaScript moderno, usamos el prefijo `#` para declarar atributos privados. Estos no pueden ser accedidos ni modificados desde fuera de la clase, evitando que el objeto quede en un estado inválido.

```
class CuentaBancaria {
  // Atributo privado (solo accesible dentro de la clase)
  #saldo;

  constructor(saldoInicial) {
    if (saldoInicial < 0) throw new Error("Saldo inicial no puede ser negativo");
    this.#saldo = saldoInicial;
    this.titular = "Juan"; // Público por defecto
  }

  // Método público que interactúa con el privado
  retirar(monto) {
    if (monto > this.#saldo) {
      console.log("Fondos insuficientes");
      return false;
    }
    this.#saldo -= monto;
    return true;
  }

  consultarSaldo() {
    return `Saldo disponible: ${this.#saldo}`;
  }
}

const miCuenta = new CuentaBancaria(1000);
// miCuenta.#saldo; // ❌ SyntaxError: Private field '#saldo' must be declared in the class
miCuenta.retirar(200); // ✅ OK
console.log(miCuenta.consultarSaldo()); // Saldo disponible: $800
```

## 4.3 Herencia

**Reutilizar lógica.** La herencia permite crear una nueva clase (hija) a partir de una existente (padre), absorbiendo sus métodos y propiedades. Usamos `extends` para heredar y `super()` para llamar al constructor del padre.

```
class Empleado {
    constructor(nombre, salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    trabajar() {
        return `${this.nombre} está trabajando.`;
    }
}

// Desarrollador hereda de Empleado
class Desarrollador extends Empleado {
    constructor(nombre, salario, lenguaje) {
        // super() llama al constructor de Empleado para inicializar nomb
        super(nombre, salario);
        this.lenguaje = lenguaje;
    }

    // Sobrescribimos el método trabajar (Polimorfismo)
    trabajar() {
        return `${this.nombre} está escribiendo código en ${this.lenguaje}
    }
}

const dev = new Desarrollador("Ana", 3000, "JavaScript");
console.log(dev.trabajar()); // Ana está escribiendo código en JavaScript
console.log(dev.salario);    // 3000 (heredado)
```

## 4.4 Polimorfismo

**Múltiples formas.** El polimorfismo permite que un mismo método se comporte de manera diferente en distintas clases hijas. En el ejemplo anterior, `trabajar()` hace algo diferente en `Desarrollador` que en `Empleado`.

```
// Función que acepta cualquier tipo de Empleado
function iniciarJornada(empleado) {
    // No importa si es Desarrollador, Gerente o Diseñador...
    // El método trabajar() se adaptará según el objeto real
    console.log(empleado.trabajar());
}

iniciarJornada(new Empleado("Luis", 2000));    // Luis está trabajando.
iniciarJornada(new Desarrollador("Ana", 3000, "JS")); // Ana está escribi
```

### Nota del Instructor:

-

#### Abstracción:

*Concéntrate en qué hace el objeto, no en cómo lo hace internamente.*

-

#### Encapsulamiento:

*Usa # para evitar que otros programadores rompan la lógica interna de tu objeto.*

-

#### Herencia:

*Pregúntate siempre "¿Es una relación es-un?" (Un Perro es un Animal). Si no es así, no heredes.*

-

#### Polimorfismo:

*Permite que tu código sea flexible. Llama al método sin preocuparte por el tipo exacto del objeto.*

## 5. Getters y Setters: Los Porteros de tu Objeto

¿Por qué no deberíamos acceder directamente a las propiedades? Porque si una propiedad es pública, otros pueden asignarle valores inválidos (ej: `cuenta.saldo = -500` ).

Los **getters** ( `get` ) permiten *leer* un valor procesado. Los **setters** ( `set` ) permiten *validar* antes de asignar.

```
class Producto {
    #precio;

    constructor(nombre, precio) {
        this.nombre = nombre;
        this.#precio = precio;
    }

    // Getter: Se accede como si fuera una propiedad, no un método
    get precio() {
        // Podemos formatear antes de mostrar
        return `$$${this.#precio.toFixed(2)}`;
    }

    // Setter: Permite validar antes de modificar el privado
    set precio(nuevoValor) {
        if (nuevoValor < 0) {
            console.error("El precio no puede ser negativo");
            return;
        }
        if (nuevoValor > 10000) {
            console.warn("Precio muy alto, aplicando descuento automático");
            this.#precio = nuevoValor * 0.9;
        } else {
            this.#precio = nuevoValor;
        }
    }
}

const laptop = new Producto("Laptop", 1000);
console.log(laptop.precio); // "$1000.00" (Getter)
laptop.precio = 12000;      // Precio muy alto, aplicando descuento autom
console.log(laptop.precio); // "$10800.00"
```



**Nota del Instructor:**

*Usar getters y setters (o métodos públicos) mantiene el*

**Encapsulamiento**

*. La regla de oro es: "No expongas tus entrañas (propiedades privadas), ofrece una interfaz controlada (getters/setters/métodos)."*

## 6. Buenas Prácticas en POO

---

Para que tu arquitectura no se convierta en un "Infierno de Objetos", sigue estas recomendaciones:

### 1. Nombramiento Claro (Semántica):

- **Clases:** Sustantivos en singular y con **PascalCase** ( `UserService` , `InvoiceFactory` ).
- **Métodos y Propiedades:** Verbos o frases en **camelCase** ( `calcularTotal()` , `getUserById` ).
- **Privados:** Prefijo `#` para los que realmente deben estar ocultos.

### 2. Una Clase, Una Responsabilidad (Principio de Responsabilidad Única):

No mezcles la lógica de conexión a la base de datos con la lógica de renderizado del HTML. Una clase `User` no debería saber cómo dibujarse a sí misma en la pantalla.

### 3. Composición sobre Herencia:

Si tienes una jerarquía de herencia muy profunda, el código se vuelve frágil. Prefiere **composición**: en lugar de "ser un", usa "tener un".

*Mala práctica:* `Coche extends Motor` (Un coche no es un motor).

*Buena práctica:* `Coche { constructor() { this.motor = new Motor(); } }` (Un coche *tiene un* motor).

### 4. Evita el Abuso de `super()` y Getters/Setters Simples:

- Si una subclase solo llama a `super()` y no añade nada nuevo, probablemente no necesitas la subclase.
- No crees getters y setters que solo asignen y retornen el valor sin lógica adicional. Para eso, usa propiedades públicas inicialmente. Añade los "porteros" solo cuando necesites validación o lógica extra.

# Conclusión: La Arquitectura como Competencia

---

Dominar la POO no solo les permite escribir mejor código; les permite **pensar como arquitectos de software**. Cuando enfrenten proyectos complejos, la capacidad de abstraer entidades, encapsular lógica y definir jerarquías claras será la diferencia entre un código que "funciona" y un sistema que es **escalable, mantenible y profesional**.

Utilicen este documento como referencia. Ahora, cuando manipulen el DOM, no piensen en "crear un div", piensen en crear una instancia de `ComponenteUI`. Cuando manejen datos de un formulario, no validen con funciones sueltas; creen una instancia de `Usuario` que sepa validarse a sí misma.

**¡Bienvenidos al mundo de la Arquitectura de Objetos!**